Activity 3: Bounded Statistical Side-Channel Artefact Study

Applying RTLF to author-provided timing data

CS6530 Applied Cryptography (Jul-Nov 2026) - Theme 13: Side-Channel and Implementation Security
Aayush Lal (CE24B033) - Sanchit Lodha (CE24B107) - Sanika Nair (CE24B108)

Status of this study

Component	Status
Authors' RTLF tool run on author-provided data (example-1, example-2)	**Done, real results** (Section 4.1)
Student numpy port validated against the authors' R output	**Done** (Section 4.2)
Type-1 error calibration (identical distributions)	**Done, simulated data** (Section 4.3)
Team-machine reproduction: R tool on example-1 (run twice), port validation, 200-repetition calibration	**Done** (Section 4.5)
R tool on example-2 on the team machine	**Pending**: the example-2 row below comes from the development-sandbox run
MatrixSSL 4.6.0 Bleichenbacher pairs B1-B5 (500,000 per vector)	**Pipeline built and tested on simulated data only.** The Zenodo dataset was not reachable from the development environment, so no MatrixSSL result is reported here (Section 6).

1. Objective and study path

This activity performs one bounded timing-leakage analysis using the authors' artefact, as required: *prescribed measurement set, prescribed statistical analysis, leak/no-leak result, interpretation of the statistical evidence, and relation to the implementation and the primary paper* [1]. Reproducing the full paper evaluation is not required. The planned main study is the MatrixSSL 4.6.0 Bleichenbacher analysis (artefact experiment E2, $\alpha = 0.009$, 500,000 measurements per vector); the evidence reported below comes from the author-provided datasets that could be obtained in our environment.

2. Environment and sources

Item	Value
Reference tool	RTLF `rtlf.R`, commit 2497d660 (21 Mar 2025), unmodified, github.com/tls-attacker/RTLF [3]. R 4.3.3 with tidyverse, optparse, jsonlite (Ubuntu 24.04 packages).
Student tool	`src/sca_rtlf/rtlf.py`: Python 3.12.3, numpy 2.4.4, pandas 3.0.2
Hardware	1 vCPU Linux cloud sandbox. Not comparable to the authors' Intel i7-6700 setup.
Data	`examples/example-1.csv` and `example-2.csv` from the RTLF repository: 30,000 measurements per class each, in the units recorded in the files (apparently clock cycles). The repository does not state how they were generated.
Parameters	$\alpha = 0.09$, $B = 10{,}000$ bootstrap iterations, deciles 10-90%, Bonferroni correction $\alpha/9$ per decile.
Planned dataset	Zenodo record 10817685 [4], `d_measurements_qualitative.zip` (about 809 MB), MatrixSSL 4.6.0 Bleichenbacher folder.

3. What is being tested and how the code works

RTLF [1] runs the implementation under two preconditions that differ only in a secret-dependent property and compares the two timing samples x and y. The null hypothesis H_0 says all nine deciles of the two distributions are equal. The test statistic is the absolute decile difference $|q^x_i - q^y_i|$. To decide how large a difference is explainable by noise alone, RTLF resamples x twice from itself and computes decile differences between the two resamples (pure noise, since both come from the same distribution), repeats this $B = 10{,}000$ times, and does the same for y. The threshold c^*_i for each decile is the $(1 - \alpha/9)$ quantile of these noise differences (the larger of the x-side and y-side values); the $\alpha/9$ is a Bonferroni correction for nine simultaneous tests. A leak is reported if any decile difference exceeds its threshold.

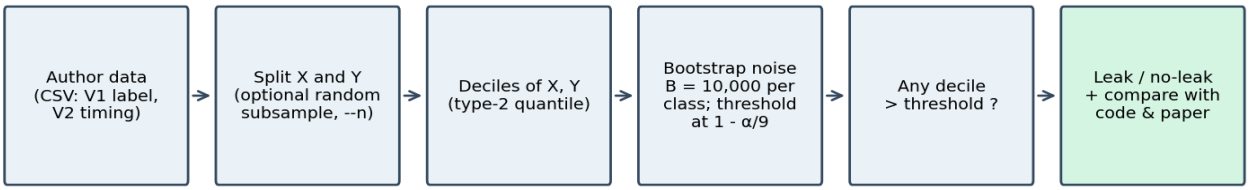


Figure 1: Analysis pipeline implemented in `scripts/run_activity3.py`. The same flow covers the authors' R tool (reference backend) and the student numpy port.

Student fast bootstrap. Brute-force resampling is too slow for 500,000 samples per class. The k-th smallest value of a resample of size n drawn from sorted data is the data value at index floor(n·U), where U is the k-th order statistic of n uniform numbers and follows a Beta(k, n-k+1) distribution. The port therefore draws one Beta variate per decile instead of resampling and sorting. This gives exact marginal distributions per decile; the thresholds depend only on these marginals, but the nine deciles are drawn independently within one resample, so the method is not byte-identical to the authors' algorithm. It was validated empirically against R (Section 4.2).

4. Results

4.1 Baseline: authors' tool on author data (α = 0.09)

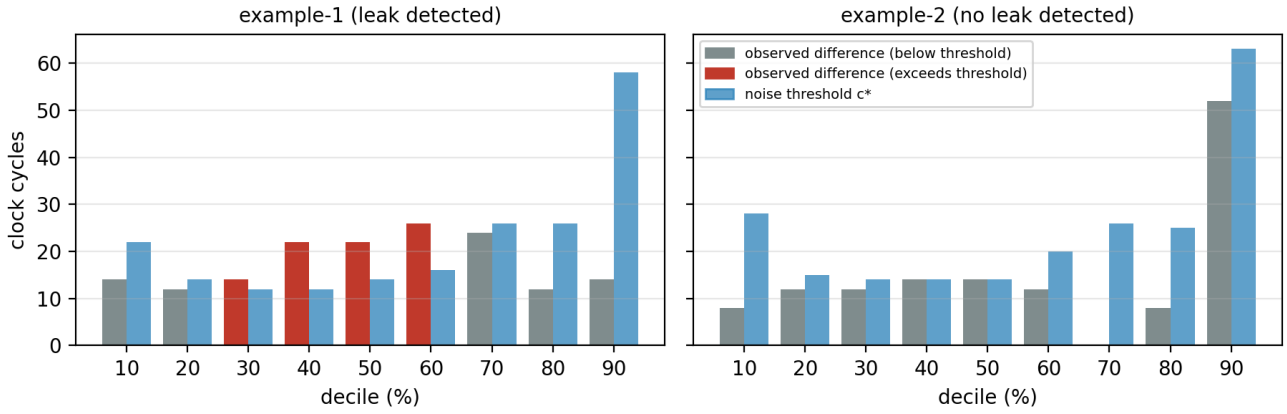


Figure 2: Observed decile differences against RTLF's bootstrap thresholds, read from the authors' `rtlf.R` output (logs in `results/`).

File	Deciles over threshold (difference vs threshold, cycles)	Decision	Exit code	Runtime
example-1	30% (14 vs 12), 40% (22 vs 12), 50% (22 vs 14), 60% (26 vs 16)	**Leak detected**	11	148 s
example-2	none; closest 40% and 50% (14 vs 14), 90% (52 vs 63)	**No leak detected**	10	150 s

For example-1 the means are 38,535 and 38,604 (a gap of about 69 cycles, under 0.2%) with standard deviations near 10,000, yet four adjacent deciles are flagged: the difference sits in the body of the distribution rather than in the tails, and is small compared with the noise. This is the kind of difference that a mean-based test handles poorly on non-normal data.

4.2 Validation of the student port

Decile differences from the numpy port are identical to R for both files, and verdicts agree. Thresholds differ by at most 2 cycles at B = 10,000 (example-1, 90% decile: 56 vs 58 cycles; example-2, 90% decile: 61-62 vs 63). The fast bootstrap takes 0.05 s per pair against about 150 s for the R tool on the same 1-vCPU machine. As a failure demonstration, running the port with a deliberately small B = 300 wrongly flagged example-2: a 99% quantile estimated from only 300 values is an unstable threshold, consistent with the paper's remark that α must stay well above 1/B [1].

4.3 Type-1 error calibration (simulated)

X and Y were drawn from the *same* skewed distribution (log-normal body plus rare spikes), analogous to the paper's same-xy case, and the test was repeated 1,000 times per cell. The authors' same-xy dataset (artefact experiment E1b) is also on Zenodo and was not available, so this is a student simulation.

α	n / class	Data	Sandbox, 1000 reps: FP (FPR) [95% Wilson CI]	Team machine, 200 reps: FP (FPR) [95% Wilson CI]
0.09	15,000	continuous	49 (4.9%) [3.7, 6.4]%	12 (6.0%) [3.5, 10.2]%
0.09	30,000	continuous	44 (4.4%) [3.3, 5.9]%	11 (5.5%) [3.1, 9.6]%
0.009	15,000	continuous	8 (0.8%) [0.4, 1.6]%	1 (0.5%) [0.1, 2.8]%
0.009	30,000	continuous	4 (0.4%) [0.2, 1.0]%	2 (1.0%) [0.3, 3.6]%
0.09	15,000	int cycles	57 (5.7%) [4.4, 7.3]%	12 (6.0%) [3.5, 10.2]%
0.09	30,000	int cycles	44 (4.4%) [3.3, 5.9]%	6 (3.0%) [1.4, 6.4]%
0.009	15,000	int cycles	2 (0.2%) [0.1, 0.7]%	0 (0.0%) [0.0, 1.9]%
0.009	30,000	int cycles	4 (0.4%) [0.2, 1.0]%	1 (0.5%) [0.1, 2.8]%

In the sandbox run all eight observed rates are at or below α. In the team's 200-repetition run seven of eight are; the exception is α = 0.009, n = 30,000, continuous data, with 2 false positives in 200 (1.0%). That is not evidence against the bound: a 95% Wilson interval for 2/200 is [0.3%, 3.6%], which contains α = 0.9%, and the 1,000-repetition run gives 0.4% for the same cell. With 200 repetitions and α = 0.009, 1 to 2 false positives are expected by chance alone. (Our first version of the calibration script printed a naive 'bound ok? NO' for this cell; the check now compares α with the interval, see Section 6.) The paper reports 9.2% for same-xy at n = 15,000 and α = 0.09 (Table 3 of [1]), slightly above α. Our simulated data differ from theirs (their same-xy mixes NOP loops with `rand()` system calls), so we do not claim to reproduce that cell.

4.4 Pipeline test on a simulated MatrixSSL-like fixture

To test `scripts/run_activity3.py` without the real archive, a fixture was generated from the early-exit structure of the paper's Algorithm 3 (B2 exits at line 3; B4 and B5 at line 14, with B5 scanning more loop iterations; B1 and B3 at line 21 and identical in instructions), with skewed noise and rare spikes, 200,000 samples per class. The pipeline reported a leak for 9 of 10 pairs, none for B1B3, and B4B5 was the weakest (7 of 9 deciles flagged), matching the code-derived expectation on 10 of 10 pairs. **This validates the pipeline logic only. It is not a measurement of MatrixSSL, and these numbers must not be cited as MatrixSSL results.**

4.5 Reproduction on the team machine

The team re-ran the study on its own Linux laptop (Python 3.12 in a virtual environment, the authors' `rtlf.R` from the cloned repository). Screenshots are in `evidence/activity3/`; the values below are transcribed from them.

Run	Outcome
R tool, example-1, α = 0.09, run 1	Leak detected; deciles 30, 40, 50, 60% flagged; thresholds 22, 13, 12, 12, 14, 16, 26, 26, 56; 141.9 s
R tool, example-1, run 2 (identical input and parameters)	Same verdict and same flagged deciles; identical decile differences; thresholds differ by 1 cycle in two deciles (20%: 13 vs 14; 90%: 56 vs 57); 140.6 s
Port vs R (B = 10,000)	Decile differences identical in both files; verdicts agree; example-1 90% threshold 56 (port) vs 58 (R)
Fast bootstrap	0.05 s per file; thresholds within 2 cycles of R; verdicts agree
Calibration, 200 reps	See Section 4.3: 7 of 8 cells at or below α; the exception is consistent with sampling noise

Runs 1 and 2 give a direct view of the bootstrap's run-to-run randomness: the observed decile differences are deterministic and identical, while the noise thresholds move by a cycle or so. Neither moves the verdict because the flagged deciles exceed their thresholds by 2 to 10 cycles. The team's second command used example-1 again (with a different output file), so a team-machine R run on example-2 is still pending.

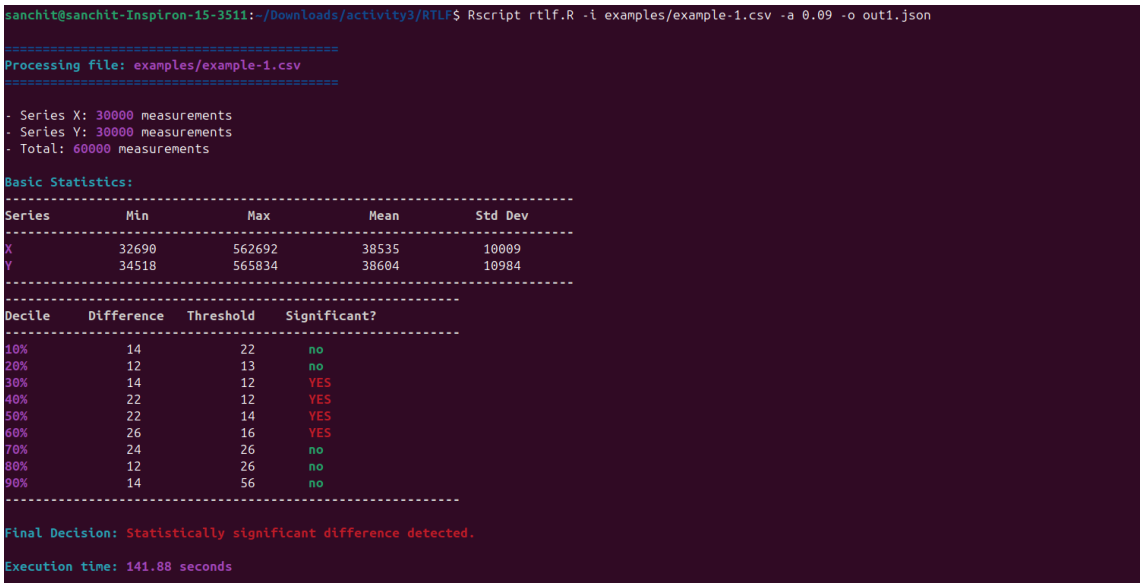


Figure 3: Team-machine run of the authors' `rtlf.R` on example-1 (screenshot 01).

5. Interpretation

A verdict is only meaningful with its parameters. example-1's leak is reported at α = 0.09, so up to 9% false alarms is built into the setting. example-2's 'no leak' means no difference was detectable at this n and noise level, not that the code is constant time; the paper states explicitly that Type-2 error cannot be bounded for arbitrarily small effects [1]. **The threshold is the noise estimate.** Thresholds of 12-63 cycles show the smallest difference this data set can resolve, and the exceedance (difference minus threshold) gives graded evidence, which the paper uses to separate probable real leaks from isolated Type-1 errors.

Relation to the implementation and the paper (for the MatrixSSL run). Algorithm 3 predicts a timing difference for every pair except B1B3, since B1 and B3 execute the same instructions. The paper's published RTLF verdicts at α = 0.009 differ from this: it flags B1B3 (a possible Type-1 error or operand-dependent timing) and does not flag B1B2, B1B5, B2B5 or B3B4 [1]. Explaining those disagreements through α, noise and sample size is the interpretation exercise the real run is meant to support, and it motivates the sample-size exploration planned for Activity 4.

6. Limitations and deviations

Limitation / deviation	Detail
Planned dataset not analysed	The development sandbox could not reach Zenodo, so the MatrixSSL B1-B5 analysis was not run. The pipeline is complete, but its filename matching was written from the authors' `collectResults.py` (re-typed as FILENAME_MAP in src/sca_rtlf/data.py) and is untested against the real archive layout. This section must be updated with real results once the pipeline is run on a machine with Zenodo access.
Bug found by team testing, fixed	Passing a placeholder path (`"<that folder>"`) to the pipeline crashed with an AttributeError because zero pair files matched. The script now validates the path, rejects placeholders, and prints a diagnostic with sample file names when nothing matches. The calibration pass/fail check was also changed from the observed rate to a Wilson interval, because a single cell was flagged by chance.
Example-2 R run	Only run in the development sandbox so far; a team-machine run is pending (Section 4.5).
Example files are generic	example-1 and example-2 are RTLF demonstration files whose generation is not documented, so no claim is made about a specific implementation.
Calibration is simulated	The E1b same-xy dataset was not available; the calibration uses student-generated data and does not reproduce the paper's Table 3.
Port is not identical	The fast bootstrap draws deciles independently and is validated against R empirically, not proven equivalent.
Runtimes	About 150 s per 30,000-per-class pair on one vCPU with the R tool; not comparable to the authors' multi-core setup.

7. Artefact provenance

Item	Origin
`rtlf.R` (commit 2497d660) and `examples/*.csv`	Author-provided, not modified
Artefact repository scripts and Zenodo dataset	Author-provided; only read. The filename-to-pair map was re-typed into the student pipeline.
`src/sca_rtlf/rtlf.py`, `scripts/run_activity3.py`, `validate_port.py`, `calibrate_type1.py`, `make_sim_fixture.py`, `fetch_dataset.sh`	Student-created
Fast Beta-order-statistic bootstrap	Student optimisation, validated against the authors' R tool
Simulated fixture and calibration data	Student-generated; not measurements of any real library

References

[1] M. Dunsche, M. Maehren, N. Erinola, R. Merget, N. Bissantz, J. Somorovsky, and J. Schwenk, "With Great Power Come Great Side Channels: Statistical Timing Side-Channel Analyses with Bounded Type-1 Errors," in *33rd USENIX Security Symposium*, 2024, pp. 6687-6704.
[2] RUB-NDS, "Artifacts: With Great Power Come Great Side Channels," GitHub repository, 2024. https://github.com/RUB-NDS/Artifacts-With-Great-Power-Come-Great-Side-Channels
[3] tls-attacker, "RTLF: R-Time-Leak-Finder," GitHub repository. https://github.com/tls-attacker/RTLF
[4] M. Dunsche et al., "USENIX'24 artifact datasets," Zenodo, Mar. 2024, doi: 10.5281/zenodo.10817685.